Praxis der Softwareentwicklung

G-Cal

Wintersemester 2025/26

Implementierungsbericht

Karlsruher Institut für Technologie (KIT)

Karlsruhe, den 20.02.2026

Karlsruher Institut für Technologie
Fakultät für Informatik
Postfach 6980
76128 Karlsruhe

Inhaltsverzeichnis

1 Einleitung **4**
1.1 Verwendete Bibliotheken 4

2 Änderungen zum Entwurf **5**
2.1 Model 5
2.1.1 LocalData 7
2.2 Server 9
2.2.1 Daten 9
2.2.2 Events 9
2.2.3 Controller 9
2.2.4 User 10
2.2.5 Datenbankentwurf 11
2.2.6 Verteilungsprozess 11
2.3 Ui 13

3 Implementierungsprozess **14**
3.1 Versionskontrolle 14
3.2 Arbeitsaufteilung 14
3.3 Planung 14

4 Unit-Tests **14**
4.1 Model 14

Abbildungsverzeichnis

1 Der Implementierungsbericht 15

1 Einleitung

Folgender Implementierungsbericht dient als Anschluss zum Entwurf, dabei wird auf die eingeführten Änderungen zum Entwurf sowie den Implementierungsprozess eingegangen.

1.1 Verwendete Bibliotheken

Folgende Bibliotheken wurden zusätzlich zum Entwurf verwendet.

JUnit

Für die Unit-Tests wurde die *JUnit 4* Bibliothek verwendet. Diese erlaubt das effiziente Testen von Komponenten des Servers und des Clients.

2 Änderungen zum Entwurf

2.1 Model

Model Facade Die Schnittstelle der *Model Facade* wurde um folgende Methoden ergänzt:

- **fun getAllGroups(): Flow<List<Group>>**
- **suspend fun updateGroup(group: Group): Result<Unit>**
- **suspend fun addFriend(username: String): Result<Unit>**
- **suspend fun removeFriend(username: String): Result<Unit>**
- **suspend fun getAllAchievements(): Result<List<Achievement>>**
- **suspend fun postSharedEvent(event: SharedEvent, users: List<User>): Result<Unit>**
  Spezifiziert zusätzlich die User, mit denen das Event geteilt werden soll.
- **suspend fun syncExperiencePoints(): Result<Unit>**
  Bietet dem ViewModel die Möglichkeit, die modellinterne XP-Verteilung zum Start eines Tages anzustoßen.

Zusätzlich wurde der Rückgabetyp der Methode `completeEvent(event: Event)` zu `Result<List<Achievement>>` geändert, um das ViewModel über potenziell erreichte Achievements zu informieren.
Intern wird das Interface nun von der Klasse `ModelData` implementiert, was die Codestruktur durch die Reduzierung der Klassenanzahl übersichtlicher gestaltet.

Request API Die Schnittstelle der *Request-API* wurde um folgende Methoden ergänzt:

- **suspend fun getLeaderboard(): Response<List<LeaderboardEntry>>**
- **suspend fun addUserGroup(newGroup: Group): Response<Unit>**
- **suspend fun updateGroup(updatedGroup: Group): Response<Unit>**
- **suspend fun deleteUserGroup(group: Group): Response<Unit>**

- **suspend fun deleteUserToDo(toDo: UserToDo): Response<Unit>**
- **suspend fun deleteUserAppointment(event: UserAppointment): Response<Unit>**
- **suspend fun addSharedEvent(appointment: UserSharedEvent, users: List<RegularUser>): Response<Unit>**
- **suspend fun updateSharedEvent(sharedEvent: UserSharedEvent): Response<Unit>**
  Hier sind insbesondere nur Updates der Completion-Details möglich.

Event/Group In den *Event-Interfaces* wurde der Rückgabetyp der Methode `eventID()` zu `Long` geändert. Grund hierfür ist, dass die lokale Datenbank diesen Datentyp am effizientesten als *Primary Key* verarbeiten kann. Aus demselben Grund wurde auch der Rückgabetyp der Methode `groupID()` zu einem `Long` geändert.

Achievements Das *Achievement-Interface* wurde mit den *Completion-Details* zusammengeführt. Dies vereinfacht die Speicherung in der lokalen Datenbank. Da für Achievements die zusätzlichen Informationen über den Abschlussstatus in der Regel ohnehin relevant sind, ergibt sich kein signifikanter Vorteil aus einem spezialisierten oder getrennten Zugriff auf diese Daten.

Achievement Handling Die Funktionalität der *Achievements* wird durch einen *AchievementHandler* und einen *AchievementEvaluator* bereitgestellt. Der *AchievementHandler* bietet dabei Methoden für Abfragen an das `AchievementDao` sowie eine Methode, welche für alle *Achievements* die Voraussetzungen prüft. Dabei wird intern die erwähnte *AchievementEvaluator*-Klasse genutzt, welche für jeden *AchievementType* eine spezifische Methode zur Überprüfung besitzt.

Serialisierung Alle Event-Implementierungen (`UserAppointment`, `UserToDo`, `UserSharedEvent`) sowie alle User-Implementierungen (`PersonalUser`, `RegularUser`, `LeaderboardEntry`) und `EventCD` sind serialisierbar. Dabei wird ein *LocalDateTimeSerializer* verwendet, um den Umgang mit `LocalDateTime` zu ermöglichen. Die Interfaces `XP` und `Group` sind nun *sealed* und serialisierbar; dabei existiert jeweils eine reguläre Implementierung und ein Null-Object.
Um bei der Erstellung von `SharedEvents` den Server über die Teilnehmer zu

informieren, existiert die serialisierbare Klasse `SharedEventRequest`, welche ein `SharedEvent` und eine Liste von Usern bündelt.

Repositories Die im Entwurf erwähnten *Observer* wurden zu Repositories umfunktioniert, namentlich `EventRepo` und `UserRepo`. Dabei starten die `observe`-Methoden stets einen Background Refresh, der die Daten vom Server in die lokale Datenbank überträgt. Parallel dazu wird der angefragte Wert direkt aus der lokalen Datenbank zurückgegeben und nachträglich aktualisiert. Zusätzlich bieten die Repositories nun die Möglichkeit, `update`, `add` und `delete` für ihre jeweiligen Entitäten (Events/User) auszuführen.

XpHandling Der `XpDistributor` wurde gemäß des Entwurfs umgesetzt. Der Maximalwert für einen Tag liegt bei 128 XP, die frühestens nach acht Stunden erreicht werden. Wegen des *Randomizers* ist jedoch stets ein niedrigerer Wert zu erwarten. To-dos werden relativ zu den an diesem Tag bereits gesammelten XP gewichtet, indem diese mit einem Faktor von 0,1, 0,15, 0,2 oder 0,25 multipliziert werden. Sollten diese Daily-XP kleiner als 50 sein, werden sie für die Berechnung auf 50 gesetzt, somit steigt die XP-Menge für To-dos erst an, wenn bereits einige *Appointments* erledigt wurden. *SharedEvents* werden anhand ihrer Dauer konstant berechnet, wobei die Länge in Minuten mit 0,3 multipliziert wird. Da diese Werte konstant sind, ist gewährleistet, dass jeder Nutzer die gleiche Anzahl an XP für ein *SharedEvent* erhält.

2.1.1 LocalData

Entities Die Änderungen in den Interfaces, wie der Wechsel der `eventID` von `String` zu `Long` sowie die Zusammenführung von *Achievement* und dessen *CompletionDetail* übertragen sich gleichermaßen auf die *Entities*. Ergänzt wurde das `EventEntity`, welches lediglich eine `eventId` enthält. Das Entity dient dazu, die `eventId` über die drei Tabellen `AppointmentEntity`, `ToDoEntity` und `SharedEventEntity` hinweg eindeutig zu halten. In diesen drei Tabellen existieren jeweils Fremdschlüssel auf die `EventEntity`-Tabelle. Ebenso verfügen die `EventCompletionEntities` über Fremdschlüssel, die auf die `EventEntity` verweisen.

Relationen Um die Gesamtdaten eines Events (`Event`, `EventCompletion`, potenziell `Group`) zu bündeln, nutzen wir die Klassen `AppointmentRelation`,

`ToDoRelation` und `SharedEventRelation`. Dadurch können wir sehr einfach die Gesamtdaten eines Events abfragen.

Dao `EventDao` und `UserDao` sind jetzt abstrakte Klassen, da einige Methoden hinzugekommen sind, welche nur intern verwendet werden und somit mit `protected` deklariert werden sollten.

- **EventDao**
  Die `observe...InFull`-Methoden geben die jeweiligen Relationen zurück. Die Erstellung von To-dos und *Appointments* in der Datenbank wird über `insertNewAppointment()` bzw. `insertNewTodo()` geregelt, wobei zuerst eine neue valide ID in `EventEntity` generiert und diese danach an die *Appointment*/*Todo*- und deren `CompletionEntity` übergeben wird. *SharedEvents* werden nur vom Server aus erstellt und über eine `upsert`-Methode verwaltet, welche die `eventId` vom Server übernimmt, da diese für jeden Teilnehmer identisch sein muss. Zusätzlich wurden im Entwurf fehlende `update`/`delete`-Methoden ergänzt.

- **UserDao**
  Die `refreshLeaderboard()`-Methode wird intern durch `clearLeaderboard()` und `insertLeaderboard()` umgesetzt.

- **AchievementDao**
  Das `AchievementDao` wurde um spezifischere Methoden wie `getCompletedAchievementsIn(start: LocalDateTime, end: LocalDateTime)` ergänzt, welche hauptsächlich für den Bericht benötigt werden.

LocalData `LocalData` bleibt, abgesehen von den neuen Relationen und *Entities*, dem Entwurf treu.

2.2 Server

Am Server-Entwurf wurden einige Änderungen vorgenommen.

2.2.1 Daten

Die Entscheidung, Datentypen wie IDs in Wrapper Klassen zu modellieren, hat sich zwar durchaus aus sinnvoll gezeigt, allerdings ist der damit verbundene Aufwand bezüglich der Serialisierung von Objekten nicht lohnenswert. Da die IDs und Nutzernamen durch das Model des Clients vergeben werden, ist eine validierung des Formats nicht notwendig.

Ähnlich wie im Client wurde die Wrapper-Klasse *ExperiencePoints* eingeführt, die den Erfahrungspunkte-Wert speichert. Auch hier begründet sich die Entscheidung in der einfacheren Serialisierung. Aus dem gleichem Grund wurde die Klasse *EventCompletionDetail* eingeführt.

Zusätzlich wurde eine Klasse *RegularUser* eingeführt, welche verhindert, dass bei der Kommunikation zwischen Client und Server zu viele unnötige Daten übertragen werden. Eine Klasse SharedEventRequest kapselt außerdem ein SharedEvent sowie die Nutzer, mit denen dieses geteilt wird.

2.2.2 Events

Um die Erstellung von Global Challenges wurde ein neuer *ChallengeManager* eingeführt. Dieser bietet die Funktionalität zur Erstellung neuer globaler Challenges, welche periodisch vom Server aufgerufen wird. Dafür wurde auch ein neuer Datentyp *Challenge* eingeführt, welcher das *SharedEvent* sowie einen minimalen Erfahrungswert speichert, der notwendig ist, um an der Challenge teilzunehmen.

getGlobalChallenge() : Challenge

2.2.3 Controller

Aufgrund der verschiedenen REST-Routen für die Untertypen von Events haben wir uns entschieden, die Übersetzung in die Datenbank aufzuteilen. Dazu wurden zusätzliche Methoden eingeführt.

updateAppointment(username : String, updatedAppointment : Appointment) : Result $\langle Unit \rangle$

updateToDo(username : String, updatedToDo : ToDo) : Result $\langle Unit \rangle$

updateSharedEvent(username : String, updatedSharedEvent : SharedEvent) : Result $\langle Unit \rangle$

deleteAppointment(username : String, eventID : Long) : Result $\langle Unit \rangle$

deleteToDo(username : String, eventID : Long) : Result $\langle Unit \rangle$

deleteSharedEvent(username : String, eventID : Long) : Result $\langle Unit \rangle$

updateGroup(username : String, updatedGroup : Group) : Result $\langle Unit \rangle$

deleteGroup(username : String, groupID : Long) : Result $\langle Unit) \rangle$

Folgende Methode wird periodisch aufgerufen (momentan alle 7 Tage), welche dann eine Globale Challenge erstellt, also ein Shared Event mit allen Nutzern erstellt

createGlobalChallenge() : Result $\langle Unit \rangle$

Der EventManager und der GroupManager setzen diese Methoden ebenfalls entsprechend um, außerdem besitzt der EventManager folgende Methode, die sicherstellt, dass für ein SharedEvent eine eindeutige EventID genutzt wird, die noch keiner der User besitzt.

generateUniqueEventID() : Long

2.2.4 User

Anfragen über die Nutzerinformation wie bspw. Erfahrungspunkte sowie Änderungen von diesen wurde in den Methoden *setUserInfo()* und *getUserInfo()* zusammengefasst.

getUserInfo(username : String) : Result $\langle Account \rangle$

updateUserInfo(username : String, updatedAccount : Account) : Result $\langle Unit \rangle$

Außerdem wurde die *addUser(username : String)* Methode in den *User-Authenticator* verschoben, da dieser für die Erstellung eines Accounts zuständig ist, falls der aus dem Token in der Methode *authenticate(token : String)* extrahierte Account noch nicht in der Datenbank gespeichert ist.

2.2.5 Datenbankentwurf

Der im Entwurf vorgestellte Datenbankentwurf hat eine Vererbungsmodellierung der Event-Typen sowie eine Verbindung zu einem Event Completion Detail vorgesehen. Da die Umsetzung einer Vererbung in SQL zwar möglich, aber nicht wirklich effizient ist, haben wir uns für eine gemeinsame Entity Event entschieden, welche den Typ des Events speichert und somit eventuelle z

2.2.6 Verteilungsprozess

Auf den Verteilungsprozess wurde im Entwurf wenig eingegangen. Für die effiziente Verteilung des Servers sowie der zugehörigen Datenbank nutzen wir Docker und Docker-Compose. Dies erlaubt es, aus einem Dockerfile sowie einem docker-compose File die zwei Container zu erstellen, die einerseits den Kotlin-Server, als auch die PostgreSQL-Datenbank laufen lassen. Eine Konsequenz aus dieser Entscheidung war, dass wir davon abgesehen haben, Konfigurationsdaten aus einem Config-File zu lesen, sondern diese über ein .Env-File in die Docker Umgebungsvariablen integriert haben. Der Server-Container nutzt diese Umgebungsvariablen, um einen Ktor-Server zu starten sowie um sich mit der Datenbank zu verbinden. Diese wird mittels des anderen Containers gestartet, welcher die Datenbank mit einem gegebenen SQL-Schema initialisiert und persistent in Volumes speichert, sodass bei Löschung des Containers die Nutzerdaten weiterhin bestehen bleiben.

Eine laufende Instanz befindet sich auf der bw-Cloud, welche für die Nutzung der g-cal App vorgesehen ist. Da die zugewiesene Instanz leider nicht sehr leistungsstark ist, haben wir uns dazu entschieden, den Build Prozess aus dem Dockerfile auszuklammern und die FatJar mit den benötigten Abhängigkeiten

lokal zu bauen und auf den Server zu übertragen. Das Verhalten ist aber, wie im Installation Guide beschrieben, anpassbar.

2.3 Ui

Im ProfileScreen wurde eine Schaltfläche zur Freundesverwaltung implementiert. Diese öffnet einen modalen Dialog in dem durch Eingabe eines validen Benutzernamens eine einseitige Freundschaftsanfrage initiiert werden kann. Die Validierung des Eingabewerts erfolgt clientseitig anhand der vordefinierten Regex-Vorgaben, um die Konsistenz mit der Datenbank sicherzustellen.

Für die Erstellung und Bearbeitung von SharedEvents wurde eine Funktion zur Teilnehmerauswahl integriert. Hierbei kann der Nutzer aus einer dynamisch geladenen Liste seiner Freunde die gewünschten Teilnehmer für das gemeinsame Event selektieren.

3 Implementierungsprozess

3.1 Versionskontrolle

Als Versionskontrolle wurde Git verwendet, dazu wurde der Gitlab-Server des KIT verwendet. Zur Erstellung von Funktionen wurden Feature-Branches genutzt, die dann bei Fertigstellung in den Main-Branch gemerged wurden. Zur Kontrolle bei Änderungen wurden lokale Tests verwendet, keine CI/CD-Pipeline oder ähnliches, da dies den Umfang sprengen würde.

3.2 Arbeitsaufteilung

Die Aufteilung der Arbeit ist in ein Server-Team und ein Client-Team erfolgt, wobei sich beim Client jeweils eine Person um View, ViewModel und Model gekümmert hat.
Zur Kommunikation und zur gemeinsamen Abstimmung wurden hauptsächlich WhatsApp sowie Besprechungen über Discord genutzt.

3.3 Planung

Der in der folgenden Abbildung dargestellte Plan zeigt den geplanten Implementierungsablauf. Dabei wurden Abhängigkeiten zwischen verschieden Komponenten bestimmt, wie beispielsweise die Kommunikation zwischen Client und Server über RequestAPI und Controller.

4 Unit-Tests

4.1 Model

Es existieren Datenbank Tests, welche die Grundfunktionalität der Lokalen Room Datenbank mittels Android Test prüfen.

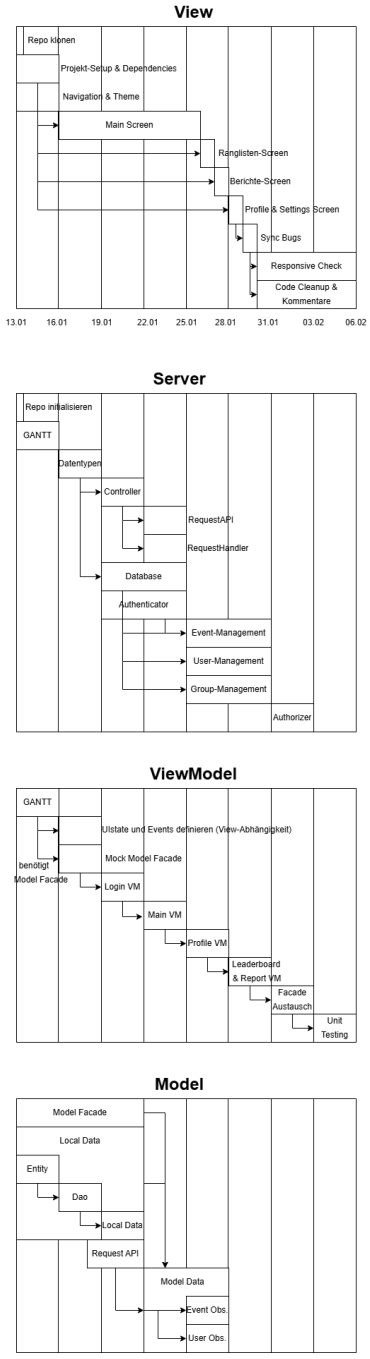

Abbildung 1: Der Implementierungsbericht